FLUTTER
CHECKLIST
By Flutter Mapp

Flutter Checklist

Table of contents

Table of contents **2**
- Master Layouts 3
- Layouts 3
- Visual Layouts 3
- User Inputs 4
- Show 5
- Builders 6
- States 6
- Navigators 7
- Data types 8
- Functions 9
- Classes 10
- Pass Data 10
- Null Safety 12
- Dart exercises 13

Other **15**
- Auto const 15
- Extensions 15
- Shortcuts 16
- Start a new project 16
- Structure 17
- Packages 17
- Assets 17
- Key concepts 19
- Change app name 19

Debug **20**
- Type 1 20
- Type 2 20

Next steps **21**
- Firebase 21
- Save data locally 21

Section 1: Widgets

Master Layouts

MaterialApp
Scaffold
SafeArea
AppBar
FloatingActionButton
NavigationBar
BottomNavigationBar
TabBarView (with TabBar and DefaultTabController)
PageView
Navigation Drawer

Layouts

SizedBox
Column
Row
Wrap
Stack
Center
Padding
SingleChildScrollView
ListView (.builder)
GridView
Spacer
Expanded
Flexible
FittedBox (Work best with Text)
RotatedBox
Align
Positioned

Visual Layouts

Container
Color ➡ Colors.blueGrey.withOpacity(0.5)

ListTile
GridTile
GridTileBar
Divider
VerticalDivider
Card
Text
Image
Icon
CircularProgressIndicator
LinearProgressIndicator
Badge
DataTable
CircleAvatar
BackdropFilter
ClipRRect

User Inputs

ElevatedButton
FilledButton
OutlinedButton
TextButton
IconButton
GestureDetector
InkWell
Dismissible
Checkbox (CheckboxListTile)
Switch (SwitchListTile)
TextField
TextFormField
Form
DropDownMenu
DropDownButton
PopUpMenuButton
RadioButton
Slider
Chip
ChoiceChip
ToggleButton
Visibility
ExpansionTile
CloseButton

Show

showBottomSheet
showDatePicker
showDateRangePicker
showDialog
showLicensePage
showModalBottomSheet
showSearch
showTimePicker
showCupertinoDialog
showCupertinoModalPopup
showSnackBar (ScaffoldMessenger)
showMaterialBanner (ScaffoldMessenger)

```
showDialog(
            context: context,
            builder: (context) {
              return const AlertDialog(
                content: Text('Dialog'),
                actions: [
                  CloseButton(),
                ],
              );
            },
          );
```

```
ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content:
                      Text('Snackbar'),
                  behavior: SnackBarBehavior.floating,
                ),
              );
```

Builders

FutureBuilder
StreamBuilder
Builder
LayoutBuilder (MediaQuery.of(context).size)
OrientationBuilder (MediaQuery.of(context).orientation)
ValueListenableBuilder (with ValueNotifier)

```
ValueNotifier<bool> isDarkNotifier = ValueNotifier(false);
```

```
FloatingActionButton(
        onPressed: () {
          isDarkNotifier.value = !isDarkNotifier.value;
        },
        child: ValueListenableBuilder(
          valueListenable: isDarkNotifier,
          builder: (context, isDark, child) {
            if (isDark) {
              return const Icon(Icons.light_mode);
            } else {
              return const Icon(Icons.dark_mode);
            }
          },
        ),
      )
```

States

Stateless (Can’t refresh the screen)
Stateful (Can refresh the screen)
SetState (Refresh the screen)

```
setState(() {
     //Do something
     });
```

Navigators

Pop
PushReplacement
Push

```
Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) {
                  return const HomePage();
                },
              ),
            );
```

Dart

Data types

Strings

```
String value = 'Hello';
```

.substring
.split
.toLowerCase
.toUpperCase

Int

```
int value = 5;
```

.isEven
.isOdd
.toDouble
.toString
.toStringAsFixed

Double

```
double value = 5.0;
```

.round
.toInt
.toString
.toStringAsFixed

Bool

```
bool value = true;
```

Lists

```
List<bool> values = [true, false, true, false];
```

.add
.length
.generate
.elementAt

Maps

```
Map<String, bool> values = {
  'value1': true,
  'value2': false,
  'value3': true,
};
```

.addAll

Functions

Looping

```
for (var i = 0; i < 5; i++) {
    print(i);
  }
```

```
int i = 0;
  while (i < 10) {
    i++;
    doThis();
  }
```

Arguments

```
void doSomething() {
  doThis(stringValue: 'Hello');
  doThis(stringValue: 'Hello', intValue: 5);
  doThat('Hello', null);
  doThat('Hello', 5);
}
void doThis({required String stringValue, int? intValue}) {
  print(stringValue);
  print(intValue);
}
void doThat(String stringValue, int? intValue) {
  print(stringValue);
  print(intValue);
}
```

Conditions

```
void doSomething() {
  if (0 == 1) {
    doThis();
  } else {
    doThat();
  }
}
```

```
void doSomething() {
  0 == 1 ? doThis() : doThat();
}
```

Classes

```
class CardClass {
  CardClass({
    required this.title,
    required this.imagePath,
  });
  String title;
  String imagePath;
}
```

Pass Data

```
class RandomPage extends StatelessWidget {
  const RandomPage({
    super.key,
    required this.title,
    this.description,
    this.isDark = false,
    required this.function,
    required this.callback,
  });
  final String title;
  final String? description;
  final bool isDark;
  final Function(int) function;
  final VoidCallback callback;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: [
          Text(title),
          Text(description ?? 'Default'),
          Text(isDark ? 'Is dark' : 'Is light'),
          TextButton(
            onPressed: () {
              callback();
```

```
            },
            child: const Text('Callback'),
          ),
          TextButton(
            onPressed: () {
              function(5);
            },
            child: const Text('Function'),
          ),
        ],
      ),
    );
  }
}
```

```
class RandomPage extends StatefulWidget {
  const RandomPage({
    super.key,
    required this.title,
    this.description,
    this.isDark = false,
    required this.function,
    required this.callback,
  });
  final String title;
  final String? description;
  final bool isDark;
  final Function(int) function;
  final VoidCallback callback;

  @override
  State<RandomPage> createState() => _RandomPageState();
}

class _RandomPageState extends State<RandomPage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: [
```

```
          Text(widget.title),
          Text(widget.description ?? 'Default'),
          Text(widget.isDark ? 'Is dark' : 'Is light'),
          TextButton(
            onPressed: () {
              widget.callback();
            },
            child: const Text('Callback'),
          ),
          TextButton(
            onPressed: () {
              widget.function(5);
            },
            child: const Text('Function'),
          ),
        ],
      ),
    );
  }
}
```

Null Safety

```
// The "?" means "This can be null";
// The "??" means "If it's null, then show the default value"
// The "!" means "I confirm that this value is not null"

String? value1 = 'Hello';
String? value2;
String value3 = 'Hi';

String value4 = value1!;            // value4 = 'Hello'
String value5 = value1 ?? value3;   // value5 = 'Hello'

String value6 = value2!;            // value6 = ERROR
String value7 = value2 ?? value3;   // value7 = 'Hi'
String value8 = value2 ?? value2!;  // value8 = ERROR

String value9 = value3;             // value9 = 'Hi'
```

Dart exercises

(Data types)

```dart
void main() {
  String name = 'Flutter Mapp';
  String food = 'Pizza';
  int number = 50;

  print ('My name is $name, I like $food and the best number is $number');
}
```

Console

```
My name is Flutter Mapp, I like Pizza and the best number is 50
```

(Classes, arguments, data types)

```dart
class Person {
  Person({
    required this.name,
    required this.food,
    required this.number,
  });
  String name;
  String food;
  int number;
}

void main() {
  Person person = Person(name: 'Flutter Mapp', food: 'Pizza', number: 5);

  print ('My name is ${person.name},');
  print ('I like ${person.food},');
  print ('The best number is ${person.number}.');
}
```

Console

```
My name is Flutter Mapp,
I like Pizza,
The best number is 5.
```

Documentation

(Data types)

```dart
void main() {
  double number = 2.5;
  bool isAvailable = true;
  List wordsList = ['Flutter', 'Mapp', 'YouTube'];
  Map translateMap = {
    'Monday': 'Lundi',
    'Friday': 'Vendredi',
    'Sunday': 'Dimanche',
  };

  print ('The best decimal number is $number,');
  print ('Is it available? Answer: $isAvailable');
  print ('What is the first word in the list? Answer: ${wordsList[0]}');
  print ('What is the second word in the list? Answer: ${wordsList[1]}');
  print ('What is Moday in French? Answer: ${translateMap['Monday']}');
}
```

Console

```
The best decimal number is 2.5,
Is it available? Answer: true
What is the first word in the list? Answer: Flutter
What is the second word in the list? Answer: Mapp
What is Moday in French? Answer: Lundi
```

Documentation

(Function, looping, conditions, data types)

```dart
void main() {
  List wordsList = ['Flutter', 'Mapp', 'YouTube'];
  bool? isAvailable;
  int number = 1;

  if(number>0){
    isAvailable = true;
  }else{
    isAvailable = false;
  }

  printEveryWords(list: wordsList);
  print(isAvailable ? 'Available': 'Unavailable');
}

void printEveryWords ({required List list}){
  for (var i = 0; i < list.length; i++) {
    print(list[i]);
  }
}
```

Run

Console

```
Flutter
Mapp
YouTube
Available
```

Documentation

Other

Shortcuts

Ctrl+b = Minimize/Maximize the Visual Studio Side Bar
Ctrl+j = Minimize/Maximize the Console
Alt+Shift+f = Format Document (Structure the code and make sure to add commas)
Refactor (Right Click) = Wrap with a Widget
Ctrl+space (Command+space for mac) = See all the possible arguments available
Ctrl+s = Save
Ctrl+x = Cut
Ctrl+v = Paste
Ctrl+c = Copy
Ctrl+d = Select the next identical word
Ctrl+l = Select the entire row
Ctrl+z = Undo

Start a new project

This will automatically set up your application Id, so you will save time in the future

```
flutter create --org com.yourwebsite your_app_name
```

This will not set your application Id

```
flutter create your_app_name
```

Packages

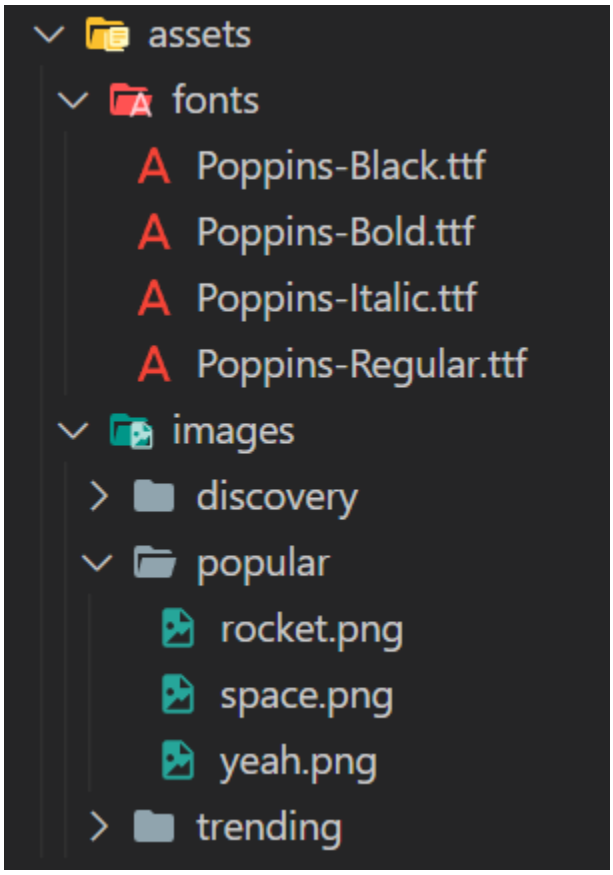
Go on https://pub.dev/
Add in dependencies or dev_dependencies

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_launcher_icons: ^0.12.0
```

Assets

Get your fonts on https://fonts.google.com/




```yaml
assets:
  - assets/images/discovery/
  - assets/images/popular/
  - assets/images/trending/

fonts:
  - family: Poppins
    fonts:
      - asset: assets/fonts/Poppins-Regular.ttf
      - asset: assets/fonts/Poppins-Bold.ttf
        weight: 700
      - asset: assets/fonts/Poppins-Black.ttf
        weight: 900
      - asset: assets/fonts/Poppins-Italic.ttf
        style: italic
```

Key concepts

Flutter is Widget inside Widget

- Add a Widget
- Add an argument in the Widget (Ctrl+space) (Command+space for mac)
- Add another Widget in the argument
- Do this forever (Until your app is done)

Widgets **always** start with a **capital letter.**
Arguments **always** start with a **lowercase letter.**

In this example (Read carefully the capital and lowercase letters):

- **S**caffold is a Widget
- **a**ppBar is an argument of the **S**caffold Widget
- **A**ppBar is a Widget placed inside the **a**ppBar argument.
- **t**itle is an argument of the **A**ppBar Widget
- **T**ext is a Widget placed inside the **t**itle argument.

```
return Scaffold(
  appBar: AppBar(
    title: const Text('Mapp blog'),
  ), // AppBar
```

Remember, it's Widget inside Widget!
Use this to see all the possible arguments available (Ctrl+space) (Command+space for mac)

Change app name

552

Android

Open `AndroidManifest.xml` (located at `android/app/src/main`)

```
<application
    android:label="App Name" ...> // Your app name here
```

iOS

Open `info.plist` (located at `ios/Runner`)

```
<key>CFBundleDisplayName</key>
<string>App Name</string> // Your app name here
```

Debug

Type 1

If you have an error like this (With red curly lines):

```
ElevatedButton(
  onPressed: () {},
```

- Put your mouse over the Widget with the red curly line.
- A box will appear, scroll down at the bottom of the box
- Read the error
- This is what you should see:

```
  required Widget? child,
})
package:flutter/src/material/elevated_button.dart
Create an ElevatedButton.
The [autofocus] and [clipBehavior] arguments must not be null.
Expected to find ','. dart(expected_token)
View Problem (Alt+F8)    No quick fixes available
ElevatedButton(
  onPressed: () {},
```

Type 2

You can also have hidden errors (Sometime the screen will be red)

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE                    Filter (e.g. text, !exclude)
'package:flutter/src/widgets/container.dart':                   package:flutter/…/widgets/container.dart:1
Failed assertion: line 273 pos 15: 'color == null || decoration == null'

The relevant error-causing widget was
MyHomePage                                                      package:my_first_app/main.dart:27
When the exception was thrown, this was the stack
#2      new Container                                           package:flutter/…/widgets/container.dart:273
#3      _MyHomePageState.build                                  package:my_first_app/main.dart:101
#4      StatefulElement.build                                   package:flutter/…/widgets/framework.dart:4992
#5      ComponentElement.performRebuild                         package:flutter/…/widgets/framework.dart:4878
#6      StatefulElement.performRebuild                          package:flutter/…/widgets/framework.dart:5050
#7      Element.rebuild                                         package:flutter/…/widgets/framework.dart:4604
```

- Open the Debug Console in the terminal
- Click the link on the top and it will bring you to your error in the code
- If you can’t understand, google search the red error

Next steps

Firebase

Authenticate users
Create a database

Save data locally

Use this: https://pub.dev/packages/shared_preferences